Họ và tên: Lê Quốc Khánh **Mã số sinh viên:** 24161276 **Lớp:** 24161TKVM1B

BÀI GIẢI CÂU HỎI ÔN TẬP
BIÊN DỊCH VÀ THỰC THI TRÊN ARM CORTEX-M

Hệ thống sử dụng STM32F103C8 lõi ARM Cortex-M3

CÂU 1: QUÁ TRÌNH TỪ CHƯƠNG TRÌNH C SANG CHƯƠNG TRÌNH TRÊN CHIP (2 ĐIỂM)

a. Trình bày bốn giai đoạn mà bộ công cụ arm-none-eabi-gcc thực hiện để chuyển main.c và cambien.c thành một tệp thực thi .elf. Nêu rõ đầu vào và đầu ra (dạng tệp) của từng giai đoạn.

Mỗi tệp nguồn C được xử lý độc lập qua ba giai đoạn đầu. Sau đó, các tệp đối tượng và mã hỗ trợ mới được ghép lại ở giai đoạn liên kết.

Giai đoạn	Công việc chính	Đầu vào	Đầu ra
1. Tiền xử lý	Mở rộng các chỉ thị #include và #define; lựa chọn hoặc loại bỏ các nhánh #if, #ifdef; xóa chú thích. Chưa sinh mã máy.	main.c, cambien.c và các tệp tiêu đề .h	main.i, cambien.i
2. Biên dịch	Phân tích cú pháp và kiểu dữ liệu, tối ưu hóa, rồi chuyển mã C đã tiền xử lý thành hợp ngữ Thumb/Thumb-2 phù hợp Cortex-M3.	main.i, cambien.i	main.s, cambien.s
3. Hợp dịch	Dịch lệnh hợp ngữ thành mã máy và tạo tệp đối tượng có thể tái định vị. Tệp .o chứa section, bảng ký hiệu và thông tin relocation nhưng chưa có địa chỉ cuối cùng.	main.s, cambien.s; startup.s nếu được hợp dịch riêng	main.o, cambien.o, startup.o

Giai đoạn	Công việc chính	Đầu vào	Đầu ra
4. Liên kết	Ghép các tệp .o với startup code và thư viện; giải quyết tham chiếu chéo; đặt section vào Flash/SRAM theo linker script; gán địa chỉ tuyệt đối và điểm vào chương trình.	Các tệp .o, thư viện và tệp .ld	Tệp thực thi chương_trình.elf

Sau bốn giai đoạn, arm-none-eabi-objcopy có thể trích từ .elf thành .bin hoặc .hex để nạp vào Flash. Đây là bước chuyển định dạng tệp nạp, không phải một trong bốn giai đoạn biên dịch nêu trên.

b. Tệp kịch bản liên kết (.ld) giữ vai trò gì mà nếu thiếu nó thì chương trình không thể chạy được trên chip, trong khi biên dịch cho máy tính cá nhân lại gần như không bao giờ cần quan tâm tới tệp này?

Linker script là bản mô tả bộ nhớ của chip và là quy tắc bố trí chương trình. Với STM32F103C8, nó khai báo vùng Flash 64 KB bắt đầu ở 0x0800 0000 và vùng SRAM 20 KB bắt đầu ở 0x2000 0000. Trình liên kết không thể tự suy ra các giới hạn phần cứng này từ mã C.

Tệp .ld quy định section nào đặt ở vùng nào, thứ tự và căn chỉnh của các section, vị trí bảng vector ngắt, điểm vào Reset_Handler, đỉnh ngăn xếp và các ký hiệu biên như _sidata, _sdata, _edata, _sbss, _ebss, _estack. Các ký hiệu đó được startup code dùng để sao chép .data và xóa .bss.

Nếu bố trí sai, CPU có thể đọc hai từ đầu của bảng vector ở vị trí không đúng, mã lệnh có thể vượt Flash, dữ liệu có thể vượt SRAM, hoặc Reset_Handler không biết vùng cần khởi tạo. Tệp vẫn có thể liên kết về mặt cú pháp nhưng chương trình không khởi động đúng trên chip.

Trên máy tính cá nhân, hệ điều hành cung cấp bộ nạp chương trình, không gian địa chỉ ảo và quy ước tệp thực thi chuẩn. Trình liên kết còn có linker script mặc định đi kèm nền tảng, nên lập trình viên ứng dụng hiếm khi phải tự chỉ ra địa chỉ vật lý. Vi điều khiển bare-metal không có bộ nạp của hệ điều hành, vì vậy toàn bộ bản đồ Flash/SRAM phải được xác định ngay khi liên kết.

c. Sau khi liên kết, ba biến THONG_BAO, soLanDo và nguongCanhBao trong đoạn mã trên được xếp vào ba section khác nhau. Hãy chỉ ra tên từng section, giải thích vì sao trình liên kết lại phân loại như vậy, và cho biết mỗi section chiếm chỗ trong Flash, trong SRAM, hay trong cả hai.

Đối tượng	Section đầu ra	Lý do phân loại	Nơi chiếm chỗ
THONG_BAO	.rodata	Là mảng const, nội dung chỉ đọc và đã biết khi biên dịch. Với tùy chọn -fdata-sections, tên section đầu vào có thể chi tiết hơn như .rodata.THONG_BAO, nhưng linker vẫn gom vào .rodata.	Chỉ Flash: CPU đọc trực tiếp từ Flash; không cần bản sao trong SRAM.
soLanDo	.bss	Là biến toàn cục có giá trị ban đầu bằng 0. Chỉ cần cấp vùng chạy và xóa về 0 khi khởi động; không cần lưu 4 byte số 0 trong ảnh nạp.	SRAM khi chạy. Không có dữ liệu khởi tạo chiếm chỗ trong ảnh nạp Flash, ngoài thông tin mô tả section trong ELF.
nguongCanhBao	.data	Là biến toàn cục có giá trị khởi tạo khác 0, cụ thể là 750. Giá trị ban đầu phải được lưu trong ảnh chương trình và biến phải cho phép ghi khi chạy.	Cả Flash và SRAM: Flash giữ ảnh khởi tạo 750; SRAM giữ bản làm việc của biến.

Hàm binhPhuong() và mã máy của main() thuộc .text trong Flash, nhưng không nằm trong ba biến mà đề yêu cầu phân loại.

d. Giải thích cặp khái niệm địa chỉ nạp (LMA - Load Memory Address) và địa chỉ chạy (VMA - Virtual Memory Address). Section nào trong ý c có LMA khác VMA, và điều đó dẫn tới hệ quả gì cho đoạn mã khởi động?

LMA (Load Memory Address) là địa chỉ nơi dữ liệu của section được lưu trong ảnh chương trình, tức địa chỉ dùng để nạp hoặc giữ giá trị ban đầu. VMA (Virtual Memory Address) là địa chỉ mà CPU sử dụng để truy cập section khi chương trình đang chạy.

Section .data có LMA và VMA khác nhau: LMA nằm trong Flash, còn VMA nằm trong SRAM. Vì vậy, trước khi main() chạy, Reset_Handler phải sao chép toàn bộ ảnh khởi tạo .data từ địa chỉ LMA trong Flash sang vùng VMA trong SRAM. Nhờ đó nguongCanhBao có giá trị 750 ngay khi main() bắt đầu.

.text và .rodata được thực thi hoặc đọc trực tiếp tại Flash nên trong bố trí thông thường LMA và VMA cùng thuộc Flash. .bss chỉ có VMA trong SRAM và kích thước cần cấp; nó không có tải dữ liệu hữu ích từ Flash. Reset_Handler phải tự ghi số 0 vào toàn bộ .bss để soLanDo nhận giá trị 0.

CÂU 2: KHỞI ĐỘNG HỆ THỐNG VÀ BẢNG VECTOR NGẮT (3 ĐIỂM)

a. Ngay sau khi tín hiệu reset được giải phóng, lõi Cortex-M3 tự động đọc hai từ nhớ 32-bit đầu tiên của bảng vector ngắt. Cho biết mỗi từ nhớ đó chứa giá trị gì, và giá trị đó được nạp vào thanh ghi nào của CPU.

Ngay khi reset được giải phóng, lõi Cortex-M3 thực hiện bằng phần cứng: từ nhớ thứ nhất tại địa chỉ vector base + 0 chứa giá trị ban đầu của Main Stack Pointer và được nạp vào MSP, tức R13 đang hoạt động; từ nhớ thứ hai tại vector base + 4 chứa địa chỉ Reset_Handler kèm bit trạng thái Thumb và được dùng để thiết lập PC, tức R15, đồng thời xác lập trạng thái thực thi Thumb.

Do Flash chính đang được alias về 0x0000 0000, lõi có thể đọc hai từ tại 0x0000 0000 và 0x0000 0004, mặc dù nội dung vật lý tương ứng nằm ở đầu Flash 0x0800 0000 và 0x0800 0004.

b. Với cấu hình bộ nhớ nêu ở phần bối cảnh, hãy tính giá trị cụ thể mà từ nhớ thứ nhất phải chứa, và giải thích vì sao lại là giá trị đó.

Dung lượng SRAM 20 KB = 20 × 1024 = 20 480 byte = 0x5000 byte. Địa chỉ ngay sau byte cuối của SRAM là:

$$0x2000\ 0000 + 0x0000\ 5000 = 0x2000\ 5000.$$

Stack trên Cortex-M là full descending nên được khởi tạo ở đỉnh vùng SRAM và phát triển về địa chỉ thấp. Do đó từ nhớ thứ nhất của bảng vector phải chứa 0x2000 5000. Địa chỉ này nằm ngay trên vùng SRAM hợp lệ; lần PUSH đầu tiên sẽ giảm SP trước rồi ghi vào bên trong SRAM, ví dụ một từ 32 bit được ghi tại 0x2000 4FFC.

c. Giả sử từ nhớ thứ hai của bảng vector chứa giá trị 0x0800 0101. Hãy cho biết lệnh đầu tiên của chương trình thực sự nằm tại địa chỉ nào, và giải thích ý nghĩa của bit số 0 trong giá trị này. Điều gì sẽ xảy ra nếu bit đó bằng 0?

Bit 0 của giá trị vector bằng 1 dùng để chỉ trạng thái Thumb. Bit này là thông tin trạng thái, không phải một phần của địa chỉ vật lý vì lệnh Thumb luôn bắt đầu tại địa chỉ chẵn. Vì vậy lệnh đầu tiên của Reset_Handler thực sự nằm tại:

0x0800 0101 AND 0xFFFF FFFE = 0x0800 0100.

Nếu bit 0 bằng 0, lõi sẽ cố chuyển tới trạng thái lệnh không hợp lệ. Cortex-M3 chỉ hỗ trợ Thumb nên việc nạp một địa chỉ handler có bit 0 bằng 0 gây lỗi trạng thái thực thi, thường dẫn đến HardFault hoặc lockup ngay trong quá trình khởi động; chương trình không thể đi tới main().

d. Liệt kê theo đúng thứ tự các công việc mà hàm Reset_Handler phải hoàn thành trước khi gọi main(). Với mỗi công việc, chỉ rõ nó phục vụ cho biến nào trong đoạn mã ở phần bối cảnh.

Cần tách rõ hai phần: phần cứng Cortex-M đã nạp MSP và PC từ bảng vector trước khi Reset_Handler bắt đầu; startup code sau đó hoàn thiện môi trường chạy của ngôn ngữ C. Trong startup do STM32CubeMX sinh, trình tự điển hình là:

1. Thiết lập hoặc xác nhận MSP = _estack. Phần cứng đã nạp MSP từ vector; nhiều startup file vẫn nạp lại _estack một cách tường minh. Bước này tạo ngăn xếp hợp lệ cho lời gọi hàm và biến cục bộ. Khi main() chạy, biến cục bộ kq có thể nằm trong thanh ghi do tối ưu hóa hoặc được cấp trên stack; các lời gọi binhPhuong() cũng dựa vào stack khi cần lưu ngữ cảnh.

2. Gọi SystemInit(). Hàm này thiết lập trạng thái hệ thống mức thấp như clock và cấu hình lõi theo dự án. Nó không trực tiếp khởi tạo ba biến đã nêu, nhưng bảo đảm phần cứng ở trạng thái thích hợp trước khi chương trình ứng dụng chạy.

3. Sao chép .data từ Flash sang SRAM. Reset_Handler dùng các ký hiệu biên do linker cung cấp, thường là _sidata làm nguồn và [_sdata, _edata) làm đích. Bước này tạo giá trị ban đầu 750 cho nguongCanhBao trong SRAM.

4. Xóa toàn bộ .bss về 0. Reset_Handler ghi 0 vào vùng [_sbss, _ebss). Bước này bảo đảm soLanDo bằng 0 theo quy tắc của ngôn ngữ C mà không phải lưu các byte 0 trong Flash.

5. Khởi tạo thư viện C và các hàm khởi tạo toàn cục nếu cấu hình toolchain yêu cầu. Lệnh gọi thường gặp là __libc_init_array(). Đoạn chương trình C thuần trong đề không có constructor C++, nhưng bước này vẫn tạo môi trường runtime đúng cho một dự án tổng quát.

6. Gọi main(). THONG_BAO đã nằm trong .rodata nên có thể đọc trực tiếp từ Flash và không cần sao chép. Lúc này nguongCanhBao đã bằng 750, soLanDo đã bằng 0, stack đã sẵn sàng, nên biểu thức binhPhuong(5) có thể được thực hiện đúng.

7. Nếu main() trả về thì đi vào vòng lặp an toàn. Trong hệ bare-metal không có hệ điều hành để nhận quyền điều khiển sau main(). Tuy bước này xảy ra sau lời gọi main(), nó là phần kết thúc bắt buộc của Reset_Handler; với chương trình đề bài, vòng while (1) trong main() khiến tình huống này không xảy ra.

CÂU 3. THANH GHI PC VÀ CHU TRÌNH THỰC THI MỘT LỆNH (2 ĐIỂM)

a. Phát biểu chính xác nội dung mà thanh ghi PC (Program Counter - thanh ghi đếm chương trình, còn gọi là R15) lưu giữ tại một thời điểm bất kỳ, và mô tả ba pha của chu trình nạp lệnh - giải mã - thực thi (fetch–decode–execute).

PC hay R15 là thanh ghi điều khiển luồng lệnh trong không gian địa chỉ byte 32 bit. Xét theo luồng chương trình, nó xác định địa chỉ lệnh kế tiếp mà CPU sẽ nạp hoặc địa chỉ đích mới sau một lệnh điều khiển. Do Cortex-M3 dùng pipeline ba tầng, trong lúc một lệnh đang ở pha thực thi, các lệnh phía sau có thể đã ở pha giải mã và nạp; vì vậy giá trị PC quan sát từ mã lệnh không đơn giản là địa chỉ của lệnh đang thực thi. Trên Cortex-M3 ở trạng thái Thumb, đọc PC cho kết quả bằng địa chỉ lệnh hiện tại cộng 4 byte.

Fetch - nạp lệnh: CPU dùng luồng địa chỉ do PC chỉ ra để đọc nửa từ hoặc các nửa từ mã lệnh từ Flash qua I-bus và đưa chúng vào pipeline.

Decode - giải mã: Khối giải mã xác định đây là lệnh Thumb 16 bit hay Thumb-2 32 bit, nhận diện phép toán, thanh ghi nguồn/đích, giá trị tức thời và chế độ địa chỉ; sau đó sinh tín hiệu điều khiển.

Execute - thực thi: ALU hoặc khối truy cập bộ nhớ thực hiện phép toán; kết quả được ghi về thanh ghi hoặc bộ nhớ, các cờ cần thiết được cập nhật và PC được giữ theo luồng tuần tự hoặc thay bằng địa chỉ đích. Nếu PC đổi đột ngột, các lệnh lấy trước sai đường bị loại khỏi pipeline.

b. Khi chương trình chạy tuần tự (không có lệnh rẽ nhánh), sau mỗi lệnh thì PC tăng thêm bao nhiêu byte? Vì sao trên Cortex-M câu trả lời không phải là một con số cố định duy nhất, khác với trường hợp vi điều khiển AVR 8-bit đã học?

PC tăng theo độ dài của lệnh vừa đi qua: tăng 2 byte đối với lệnh Thumb 16 bit và tăng 4 byte đối với lệnh Thumb-2 32 bit. Cortex-M dùng tập lệnh Thumb-2 có độ dài hỗn hợp 16/32 bit nên mức tăng không cố định. Ngược lại, trong mô hình AVR 8 bit đã học, phần lớn cách trình bày dùng từ lệnh cơ sở 16 bit và PC được mô tả theo địa chỉ từ lệnh, nên luồng tuần tự thường được xem là tăng một đơn vị từ lệnh; một số lệnh AVR dài hai từ vẫn là trường hợp ngoại lệ cần lưu ý.

c. Hoàn thành bảng sau, mô tả PC bị thay đổi như thế nào trong từng tình huống:

Tình huống	Cách cập nhật PC và thông tin liên quan
Thực thi lệnh tuần tự	PC chuyển sang địa chỉ lệnh kế tiếp: địa chỉ hiện tại + 2 byte nếu lệnh dài 16 bit hoặc + 4 byte nếu lệnh dài 32 bit.
Lệnh nảy B nhan	PC được nạp bằng địa chỉ của nhãn đích; pipeline bị flush và bắt đầu nạp lại tại đích. B là nhảy không liên kết nên không tạo địa chỉ trở về trong LR.
Lệnh gọi hàm BL hàmCon	PC được nạp bằng địa chỉ hàmCon; đồng thời LR nhận địa chỉ trở về, tức địa chỉ lệnh kế tiếp sau BL với bit 0 đặt bằng 1 để duy trì trạng thái Thumb.
Lệnh trở về BX LR	CPU kiểm tra bit 0 của LR để xác định trạng thái Thumb, rồi PC nhận địa chỉ đích đã bỏ bit trạng thái, về thực chất là LR AND 0xFFFF FFFE. Pipeline được nạp lại từ địa chỉ trở về.
Xảy ra ngắt ngoại vi	Phần cứng lưu PC tiếp tục thực thi vào exception stack frame, đọc địa chỉ ISR từ vector tương ứng rồi nạp PC bằng địa chỉ handler đã bỏ bit 0; bit 0 của vector phải bằng 1. LR được đặt thành một giá trị EXC_RETURN để điều khiển quá trình trở về ngắt.

d. Trên Cortex-M3, nếu một lệnh đọc trực tiếp giá trị của PC thì kết quả nhận được lớn hơn địa chỉ của chính lệnh đang thực thi. Hãy giải thích nguyên nhân của hiện tượng này và cho biết độ lệch là bao nhiêu byte.

Nguyên nhân là pipeline ba tầng cho phép các pha nạp, giải mã và thực thi của nhiều lệnh chồng lấn. Khi lệnh tại địa chỉ A đang thực thi và đọc R15, phía nạp lệnh đã tiến lên

trước, nên giá trị PC mà kiến trúc quy định trả về là A + 4 byte. Độ lệch là 4 byte trong trạng thái Thumb, không phụ thuộc lệnh hiện tại dài 16 hay 32 bit. Riêng một số phép định địa chỉ tương đối PC, chẳng hạn tải literal, phần cứng còn căn giá trị cơ sở này xuống biên 4 byte theo quy tắc của lệnh.

CÂU 4: NGĂN XẾP VÀ CÁC ĐỊA CHỈ Ô NHỚ CỦA NÓ (3 ĐIỂM)

a. Ngăn xếp trên Cortex-M được tổ chức theo nguyên tắc FILO/LIFO và thuộc loại "đầy, hướng xuống" (full descending). Hãy giải thích cụ thể: khi thực thi lệnh PUSH, thanh ghi SP được cộng hay trừ, thao tác ghi vào ô nhớ diễn ra trước hay sau khi thay đổi SP; và trả lời tương tự cho lệnh POP.

Full nghĩa là SP luôn trỏ vào ô đang chứa phần tử trên cùng của stack; descending nghĩa là stack mở rộng về phía địa chỉ nhỏ hơn.

PUSH: với N thanh ghi 32 bit, CPU tính SP_mới = SP_cũ - 4N, sau đó lưu các thanh ghi vào vùng bắt đầu tại SP_mới. Vì vậy đây là cơ chế giảm trước rồi ghi, tương đương STMDB SP!, {Rlist}. Với một thanh ghi: SP giảm 4 byte rồi dữ liệu được ghi tại [SP].

POP: CPU đọc dữ liệu bắt đầu tại địa chỉ SP hiện tại, khôi phục các thanh ghi, rồi tính SP_mới = SP_cũ + 4N. Vì vậy đây là cơ chế đọc trước rồi tăng, tương đương LDMIA SP!, {Rlist}. Với một thanh ghi: đọc [SP] trước, sau đó SP tăng 4 byte.

Ví dụ PUSH {R0, LR} làm SP giảm 8 byte; R0 được lưu ở địa chỉ mới thấp nhất [SP] và LR ở [SP + 4]. POP một danh sách tương ứng đọc từ các địa chỉ thấp lên cao rồi tăng SP thêm 8 byte.

b. Cortex-M có hai con trỏ ngăn xếp là MSP và PSP. Nêu vai trò mặc định của từng con trỏ, cơ chế phần cứng nào quyết định con trỏ nào đang được dùng, và vì sao thiết kế này lại quan trọng khi hệ thống chạy hệ điều hành thời gian thực (RTOS).

MSP - Main Stack Pointer: được dùng mặc định sau reset. Handler mode luôn dùng MSP, nên mọi ISR và exception handler có một stack hệ thống tin cậy. Thread mode cũng dùng MSP nếu không chọn PSP.

PSP - Process Stack Pointer: thường được dùng cho mã ứng dụng hoặc từng task của RTOS trong Thread mode. PSP không được dùng để chạy Handler mode.

Chế độ thực thi và bit SPSEL trong thanh ghi CONTROL quyết định con trỏ nào được Thread mode nhìn thấy dưới tên SP: CONTROL.SPSEL = 0 chọn MSP, CONTROL.SPSEL = 1 chọn PSP. Khi exception xảy ra, phần cứng lưu khung ngoại lệ lên

stack đang hoạt động của luồng bị ngắt; handler sau đó chạy bằng MSP. Giá trị EXC_RETURN trong LR ghi lại stack nào cần dùng khi trở về.

Thiết kế hai stack đặc biệt quan trọng với RTOS: mỗi task có một vùng stack riêng và giá trị PSP của task được lưu trong khối điều khiển tác vụ, còn các exception handler như PendSV, SysTick, SVC và các ISR dùng MSP. Khi chuyển ngữ cảnh, RTOS thay giá trị PSP của task và lưu hoặc khôi phục các thanh ghi còn lại. Nhờ vậy lỗi tràn stack của một task ít có khả năng phá trực tiếp stack xử lý ngắt, đồng thời việc tách ngữ cảnh hệ thống và ngữ cảnh tiến trình trở nên rõ ràng hơn.

c. Khi một hàm C được gọi, khung ngăn xếp (stack frame) của nó thường chứa những thành phần nào? Giải thích cụ thể vì sao địa chỉ trở về bắt buộc phải được cất vào ngăn xếp trong trường hợp hàm gọi lồng nhau, trong khi với một hàm lá (leaf function - không gọi hàm nào khác) thì có thể không cần.

Tùy hàm và mức tối ưu hóa, stack frame thường chứa: các thanh ghi callee-saved mà hàm có sử dụng, thường thuộc R4-R11; bản sao LR nếu hàm còn gọi hàm khác; biến cục bộ không thể giữ hết trong thanh ghi; giá trị tạm hoặc thanh ghi bị spill; tham số vượt quá bốn tham số truyền qua R0-R3; vùng dành cho tham số khi gọi hàm con; và phần đệm để bảo đảm căn chỉnh stack, thường 8 byte tại ranh giới lời gọi theo ARM ABI.

Lệnh BL ghi địa chỉ trở về vào LR. Nếu hàm A gọi tiếp hàm B, lệnh BL B sẽ ghi đè LR đang chứa địa chỉ để A trở về hàm gọi của A. Do đó A phải cất LR cũ vào stack, thường bằng PUSH {LR}, trước khi gọi B và khôi phục nó khi kết thúc, thường bằng POP {PC} hoặc POP {LR} rồi BX LR. LIFO bảo đảm địa chỉ của lời gọi gần nhất được lấy ra trước, nên các lời gọi lồng nhau trở về đúng thứ tự.

Một leaf function không gọi hàm nào khác nên không có lệnh BL mới ghi đè LR. Nếu nó cũng không cần nhiều biến cục bộ hoặc lưu thanh ghi, trình biên dịch có thể giữ nguyên LR và kết thúc trực tiếp bằng BX LR mà không tạo stack frame. Đây là một tối ưu hóa, không phải mọi leaf function đều chắc chắn không dùng stack.

d. Khi một ngắt xảy ra, phần cứng Cortex-M tự động đẩy 8 từ nhớ 32-bit vào ngăn xếp đang hoạt động. Hãy liệt kê 8 thanh ghi đó theo đúng thứ tự địa chỉ từ thấp đến cao, cho biết SP thay đổi bao nhiêu byte, và giải thích vì sao trong danh sách đó nhất thiết phải có PC.

Khi exception được chấp nhận, phần cứng tạo basic exception stack frame trên stack đang hoạt động. Theo thứ tự địa chỉ từ thấp đến cao, tám từ 32 bit là:

Thứ tự địa chỉ	Thanh ghi	Vai trò khi khôi phục
Thấp nhất	R0	Thanh ghi tham số hoặc kết quả tạm của mã bị ngắt
2	R1	Thanh ghi tham số hoặc kết quả tạm của mã bị ngắt
3	R2	Thanh ghi tham số hoặc kết quả tạm của mã bị ngắt
4	R3	Thanh ghi tham số hoặc kết quả tạm của mã bị ngắt
5	R12	Thanh ghi tạm theo quy ước gọi hàm
6	LR	Địa chỉ liên kết của mã đang chạy trước ngắt
7	PC	Địa chỉ tiếp tục thực thi sau khi kết thúc ngắt
Cao nhất	xPSR	Cờ trạng thái, trạng thái Thumb và thông tin exception

Khung cơ bản có 8 × 4 = 32 byte, vì vậy SP giảm 32 byte khi vào ngắt và tăng lại 32 byte khi exception return. Nếu cơ chế căn chỉnh 8 byte yêu cầu chèn thêm một từ đệm, tổng mức giảm vật lý có thể là 36 byte; bit căn chỉnh trong xPSR cho phần cứng biết phải bỏ từ đệm khi khôi phục. Đáp án cơ bản mà đề yêu cầu là 32 byte.

PC bắt buộc phải có trong khung ngắt vì CPU đang rời luồng lệnh bình thường để chạy ISR. Khi handler kết thúc, cơ chế exception return phải nạp lại PC đã lưu để tiếp tục đúng tại điểm mà chương trình bị gián đoạn. Nếu không lưu PC, CPU không biết quay về địa chỉ nào, dù các thanh ghi dữ liệu khác đã được khôi phục đầy đủ.